

Agent Spec: AI/LLM Security Research Scout

Capstone agent design — upgrading the FL-04 workflow into a real agent

1. Job to be done

Given a topic (or a general “run”), the agent searches for new AI/LLM security papers and articles, decides for itself which are worth fetching in full, and produces source-grounded study notes — the same Gather → Synthesize → Draft → Review shape as FL-04, except the agent now makes the source-selection decisions I used to make by hand. This directly implements the one concrete upgrade named in the agent-vs-workflow assignment: give the Gather step direct MCP tool access instead of me manually finding and pasting each source.

2. User and usage frequency

Just me. Run manually, on demand — roughly 1–2 times a week, either when catching up on new research or before adding to the security case study on my portfolio. Not scheduled or autonomous; I decide when a run starts.

3. Tools and data needed, with access plan

Tool / data	Purpose	Access plan
alphaXiv MCP connector	Full-text arXiv paper retrieval, GitHub repo reads for code-backed papers, saving papers to my library	Already connected and tested (3 working tool calls demonstrated in the agent-vs-workflow assignment) — no new setup
Claude's built-in web search	Non-arXiv sources: vendor blogs, OWASP, industry write-ups	Already available by default in Claude.ai — no setup
Project knowledge file: “already covered” list	Prevents re-summarizing the same paper across runs	A plain text/markdown file I maintain and upload to the Project; the agent reads it, and appends to it after each run pending my confirmation

4. Draft instructions

You are my AI/LLM security research scout. When I give you a topic or say “run”: (1) search arXiv via alphaXiv and the web for sources from roughly the last 3 months relevant to the topic; (2) check each candidate's abstract before deciding whether it's worth full-text retrieval — don't fetch everything; (3) skip anything already in my ‘already covered’ list; (4) for each selected source, produce a study note using the Gather → Synthesize → Draft → Review pattern; (5) in Review, explicitly flag any statistic that traces to a different underlying study than the one being summarized, and flag preprints vs. peer-reviewed status; (6) never fabricate a citation, number, or methodology detail not actually present in fetched text — if only an abstract was accessible, say so plainly; (7) stop and ask before saving anything to my alphaXiv library or updating the ‘already covered’ list; (8) cap each run at 5 sources unless I explicitly ask for more.

5. Eval cases (defined before building)

1. Given a specific known topic (e.g. “MCP tool poisoning”), does it find at least 2 genuinely new sources not already in the covered list?
2. Given a topic where only paywalled/abstract-only sources exist, does it correctly say so instead of inventing methodology detail? (mirrors the Run 5 failure mode already found in FL-04)
3. Given a source already logged as covered, does it correctly skip re-summarizing it?
4. Given a source repeating a statistic that actually originates from a different study (like FL-04's '90% RAG poisoning' case), does the Review step catch and flag the misattribution?
5. Does it stop and ask for confirmation before calling the save-to-library tool, rather than saving automatically?
6. Given a request for more than 5 sources in one run, does it stop at 5 and ask before continuing, rather than silently exceeding the cap?

6. Risks and guardrails

Must confirm before:

- Saving any paper to my alphaXiv library (a state-changing action)
- Updating the ‘already covered’ project knowledge file
- Running more than 5 sources in a single pass

Must never:

- Present an abstract-only summary as if it were a full-text summary without disclosing the access level
- Invent a citation, statistic, or methodology detail not actually present in fetched text
- Treat an unreviewed preprint's claims as settled fact without flagging its status

7. Platform choice and justification

Chosen: Claude Project with the alphaXiv MCP connector + built-in web search. Zero new setup — the connector is already authenticated and already demonstrated working across 3 real tool calls. Free, and matched to my actual skill level and the ~10-hour budget: nearly all the build time goes into writing and testing the instructions above, not standing up new infrastructure.

Alternative considered: an n8n agent workflow. This would allow the agent to run on a schedule without me manually saying “run,” which is a real advantage. Rejected for this build: it adds a new tool, a new hosting/auth setup, and workflow-graph debugging on top of an already tight time budget — the same reasoning as the earlier Three Roads decision (free, beginner-matched, must actually finish in the available time). Manual triggering is an acceptable tradeoff at this scope; scheduled runs are a reasonable next step once the manual version is proven reliable.